

1.9 More exercises

Example 14. Define an appropriate language and formalize the following sentences using First Order Logic formulas.

1. *Bill has at least one sister.*
2. *Bill has no sister.*
3. *Bill has at most one sister.*
4. *Bill has (exactly) one sister.*
5. *Bill has at least two sisters.*
6. *Every student takes at least one course.*
7. *Only one student failed Geometry.*
8. *No student failed Geometry but at least one student failed Analysis.*
9. *Every student who takes Analysis also takes Geometry.*

1. $\exists x. \text{SisterOf}(x, \text{Bill})$
2. $\neg \exists x. \text{SisterOf}(x, \text{Bill})$
3. $\forall x \forall y. (\text{SisterOf}(x, \text{Bill}) \wedge \text{SisterOf}(y, \text{Bill}) \rightarrow x = y)$
4. $\exists x. (\text{SisterOf}(x, \text{Bill}) \wedge \forall y. (\text{SisterOf}(y, \text{Bill}) \rightarrow x = y))$
5. $\exists x \exists y. (\text{SisterOf}(x, \text{Bill}) \wedge \text{SisterOf}(y, \text{Bill}) \wedge \neg(x = y))$
6. $\forall x. (\text{Student}(x) \rightarrow \exists y. (\text{Course}(y) \wedge \text{Takes}(x, y)))$
7. $\exists x. (\text{Student}(x) \wedge \text{Failed}(x, \text{Geometry}) \wedge \forall y. (\text{Student}(y) \wedge \text{Failed}(y, \text{Geometry}) \rightarrow y = x))$
8. $\neg \exists x. (\text{Student}(x) \wedge \text{Failed}(x, \text{Geometry}) \wedge \neg \text{Failed}(x, \text{Analysis}))$
9. $\neg \exists x. (\text{Student}(x) \wedge \text{Failed}(x, \text{Geometry})) \wedge \exists x. (\text{Student}(x) \wedge \text{Failed}(x, \text{Analysis}))$
10. $\forall x. (\text{Student}(x) \wedge \text{Takes}(x, \text{Analysis}) \rightarrow \text{Takes}(x, \text{Geometry}))$

Example 15. Consider the following sentences:

1. *All actors and journalists invited to the party are late.*
2. *There is at least a person who is on time.*
3. *There is at least an invited person who is neither a journalist nor an actor.*

Formalize the sentences and prove that 3. is not a logical consequence of 1. and 2.

Solution.

1. $\forall x \cdot ((a(x) \vee j(x)) \wedge i(x) \rightarrow l(x))$
2. $\exists x \cdot \neg l(x)$
3. $\exists x \cdot (i(x) \wedge \neg a(x) \wedge \neg j(x))$

It's sufficient to find an interpretation $\mathcal{I}$ for which the logical consequence does not hold:

	$l(x)$	$a(x)$	$j(x)$	$i(x)$
<i>Bob</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>
<i>Tom</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>T</i>
<i>Mary</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>

Example 16. Let $\{c_1, \dots, c_k\}$ be a non empty and finite set of colors. A partially colored directed graph is a structure $\langle N, R, C \rangle$ where

- N is a non empty set of nodes
- R is a binary relation on N
- C associates colors to nodes (not all the nodes are necessarily colored, and each node has at most one color)

Provide a first order language and a set of axioms that formalize partially colored graphs. Show that every model of this theory correspond to a partially colored graph, and vice-versa. For each of the following properties, write a formula which is true in all and only the graphs that satisfies the property:

1. connected nodes don't have the same color
2. the graph contains only 2 yellow nodes
3. starting from a red node one can reach in at most 4 steps a green node
4. for each color there is at least a node with this color
5. the graph is composed of $|C|$ disjoint non empty subgraphs, one for each color

Solution. Language

- a binary predicate edge, where $\text{edge}(n, m)$ means that node n is connected to node m
- a binary predicate color, where $\text{color}(n, x)$ means that node n has color x

- the following constants: yellow, green, red

Axioms

0. "each node has at most one color"

$$\forall n \forall x. (\text{color}(n, x) \rightarrow \neg \exists y. (y \neq x \wedge \text{color}(n, y)))$$

1. "connected nodes do not have the same color"

$$\forall n \forall m \forall x. (\text{edge}(n, m) \wedge \text{color}(n, x) \rightarrow \neg \text{color}(m, x))$$

2. "the graph contains only two yellow nodes"

$$\begin{aligned} \exists n \exists n'. (& \text{color}(n, \text{yellow}) \wedge \text{color}(n', \text{yellow}) \wedge n \neq n' \wedge \\ & \forall m. (m \neq n \wedge m \neq n' \rightarrow \neg \text{color}(m, \text{yellow}))) \end{aligned}$$

3. "starting from a red node one can reach in at most 4 steps a green node"

$$\begin{aligned} \forall n (& \text{color}(n, \text{red}) \rightarrow \\ & (\exists n_1 \cdot (\text{edge}(n, n_1) \wedge \text{color}(n_1, \text{green}))) \vee \\ & \exists n_1, n_2 \cdot (\text{edge}(n, n_1) \wedge \text{edge}(n_1, n_2) \wedge \text{color}(n_2, \text{green}))) \vee \\ & \exists n_1, n_2, n_3 \cdot (\text{edge}(n, n_1) \wedge \text{edge}(n_1, n_2) \wedge \text{edge}(n_2, n_3) \wedge \text{color}(n_3, \text{green}))) \vee \\ & \exists n_1, n_2, n_3, n_4 \cdot (\text{edge}(n, n_1) \wedge \text{edge}(n_1, n_2) \wedge \text{edge}(n_2, n_3) \wedge \text{edge}(n_3, n_4) \wedge \text{color}(n_4, \text{green}))) \end{aligned}$$

4. "for each color there is at least a node with this color"

$$\forall x \exists n. \text{color}(n, x)$$

5. "the graph is composed of $|C|$ disjoint non empty subgraphs, one for each color"

$$\begin{aligned} & \forall x \exists n. \text{color}(n, x) \wedge \\ & \forall n \exists x. \text{color}(n, x) \wedge \\ & \forall n \forall x. (\text{color}(n, x) \rightarrow \neg \exists y. (y \neq x \wedge \text{color}(n, y))) \wedge \\ & \forall n \forall m \forall x. (n \neq m \wedge \text{color}(n, x) \wedge \text{color}(m, x) \rightarrow \\ & \left(\text{edge}(n, m) \bigvee_{i=1}^{|N|} \left(\exists n_1, \dots, n_i. \left(\text{edge}(n, n_1) \bigwedge_{j=1}^{i-1} \text{edge}(x_j, x_j + 1) \wedge \text{edge}(n_i, m) \right) \right) \right) \end{aligned}$$

Example 17. Minesweeper is a single-player computer game invented by Robert Donner in 1989. The object of the game is to clear a minefield without detonating a mine. The game screen consists of a rectangular field of squares. Each square can be cleared, or uncovered, by clicking on it. If a square that contains a mine is clicked, the game is over. If the square does not contain a mine, one of two things can happen: (1) A number between 1 and 8 appears indicating the amount of adjacent (including diagonally-adjacent) squares containing mines, or (2) no number appears; in which case there are no mines in the adjacent cells. An example of game situation is provided in the following figure: Provide a first order language that allows to formalize the knowledge of a player in a game state. In such a language you should be able to formalize the following knowledge:

1. there are exactly n mines in the minefield
2. if a cell contains the number 1, then there is exactly one mine in the adjacent cells.



3. show by means of deduction that there must be a mine in the position (3,3) of the game state of picture 3.1

Suggestion: define the predicate $Adj(x, y)$ to formalize the fact that two cells x and y are adjacent

Solution. Language

1. A unary predicate mine, where mine(x) means that the cell x contains a mine
2. A binary predicate adj, where adj(x, y) means that the cell x is adjacent to the cell y
3. A binary predicate contains, where contains(x, n) means that the cell x contains the number n

Axioms

1. There are exactly n mines in the game.

$$\exists x_1, \dots, \exists x_n \left(\bigwedge_{i=1}^n \text{mine}(x_i) \wedge \forall y \left(\text{mine}(y) \rightarrow \bigvee_{i=1}^n y = x_i \right) \right)$$

2. If a cell contains the number 1, then there is exactly one mine in the adjacent cells.

$$\forall x. (\text{contains}(x, 1) \rightarrow \exists z. (\text{adj}(x, z) \wedge \text{mine}(z) \wedge \forall y. (\text{adj}(x, y) \wedge \text{mine}(y) \rightarrow y = z)))$$

3. Show by means of deduction that there must be a mine in the position (3, 3)

from Picture 3.1 we have:

- a. $\text{contains}((2, 2), 1)$
- b. $\neg \text{mine}((1, 1)) \wedge \neg \text{mine}((1, 2)) \wedge \neg \text{mine}((1, 3))$
- c. $\neg \text{mine}((2, 1)) \wedge \neg \text{mine}((2, 2)) \wedge \neg \text{mine}((2, 3))$
- d. $\neg \text{mine}((3, 1)) \wedge \neg \text{mine}((3, 2))$

we can deduce:

$$e. \exists z. (\text{adj}((2, 2), z) \wedge \text{mine}(z) \wedge \forall y. (\text{adj}((2, 2), y) \wedge \text{mine}(y) \rightarrow y = z)) \rightsquigarrow$$

from a. and axiom 2

$$f. \text{mine}((1, 1)) \vee \text{mine}((1, 2)) \vee \text{mine}((1, 3)) \vee \text{mine}((2, 1)) \vee \text{mine}((2, 2)) \vee \text{mine}((2, 3)) \vee \text{mine}((3, 1)) \vee \text{mine}((3, 2)) \vee \text{mine}((3, 3)) \rightsquigarrow \text{from } e.$$

$$g. \text{mine}((3, 3)) \rightsquigarrow \text{from } b., c., d. \text{ and } f.$$

•

Example 18. Formalize in first order logic the train connections in Italy. Provide a language that allows to express the fact that a town is directly connected (no intermediate train stops) with another town, by a type of train (e.g., intercity, regional, interregional). Formalize the following facts by means of axioms:

1. There is no direct connection from Rome to Trento
2. There is an intercity from Rome to Trento that stops in Firenze, Bologna and Verona.
3. Regional trains connect towns in the same region
4. Intercity trains don't stop in small towns.

Solution. We define the language as follows

Constants $RM, FI, BO, VR, TN, \dots$ are identifiers of the towns of Roma, Firenze, Bologna, Verona, Trento, . . . and $InterCity, Regional, \dots$ are the identifiers of the type of trains

Predicates $Train$ with arity equal to 1, where $Train(x)$ means x is a train Town with arity equal to 1, where $Town(x)$ means x is a town

$SmallTown$ with arity equal to 1, where $SmallTown(x)$ means x is a small town

TrainType with arity equal to 2 , where *TrainType* (x, y) means that the train x is of type y .

IsInRegion with arity equal to 2 , where *IsInRegion* (x, y) means that the town x is in region y . *DirectConn* with arity equal to 3, where *DirectConn* (x, y, z) means that the train x directly connects (with no intermediated stops) the towns y and z .

Background axioms With these set of axioms we have to formalize some background knowledge which is necessary to make the formalization more adequate

1. a train has exactly one train type;

$$\forall x(\text{Train}(x) \rightarrow \exists y(\text{TrainType}(x, y))) \wedge \forall xyz(\text{TrainType}(x, y) \wedge \text{TrainType}(x, z) \rightarrow y = z) \quad (3.1)$$

2. Intercity type is different from regional type:

$$\neg(\text{InterCity} = \text{Regional}) \quad (\text{also written as } \text{InterCity} \neq \text{Regional}) \quad (3.2)$$

3. A town is associated to exactly one region

$$\forall x(\text{Town}(x) \rightarrow \exists y(\text{IsInRegion}(x, y))) \wedge \forall xyz(\text{IsInRegion}(x, y) \wedge \text{IsInRegion}(x, z) \rightarrow y = z) \quad (3.3)$$

Bibliography

[Giunchiglia and Fumagalli(2017a)] Fausto Giunchiglia and Mattia Fumagalli. Course on mathematical logics: A booklet with exercises. <https://disi.unitn.it/ldkr/ml2017/ExercisesBooklet.pdf>, 2017a. [Accessed 25-02-2026].

[Giunchiglia and Fumagalli(2017b)] Fausto Giunchiglia and Mattia Fumagalli. Course on mathematical logics: Set theory lecture. <https://disi.unitn.it/ldkr/ml2017/slides/01.Set-Theory.2017.r03.pdf>, 2017b. [Accessed 25-02-2026].

[Piff(1991)] Mike Piff. Discrete mathematics: an introduction for software engineers. Cambridge University Press, 1991.

[Rosen(2007)] Kenneth H Rosen. Discrete mathematics and its applications sixth edition. McGraw-hill, 2007.